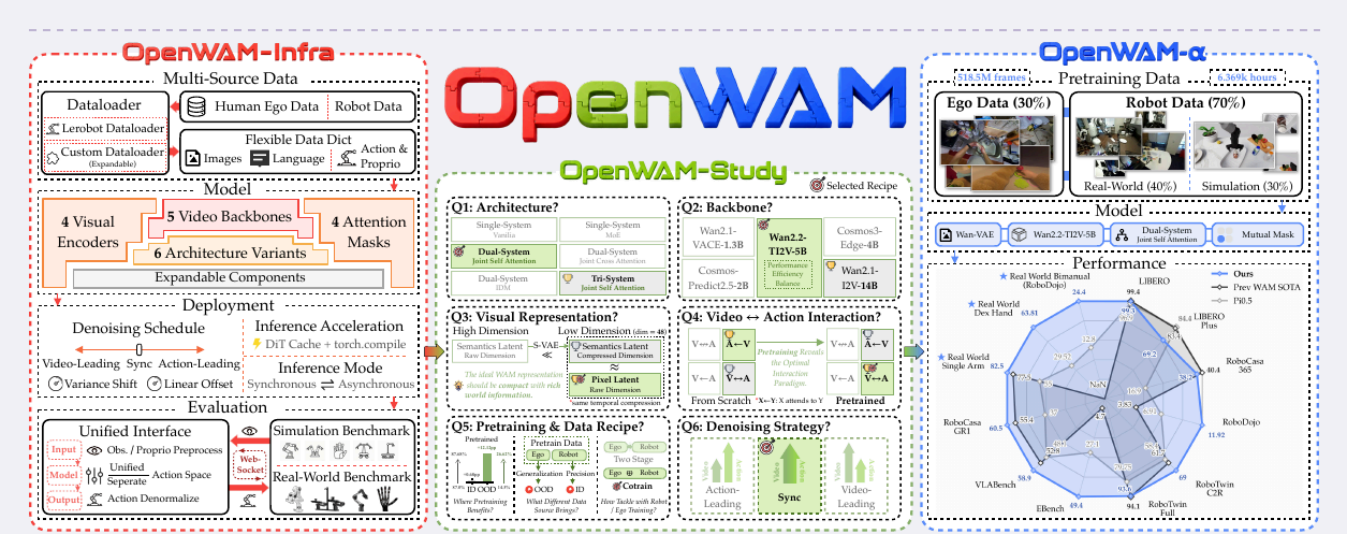
OpenWAM: 분석 리포트

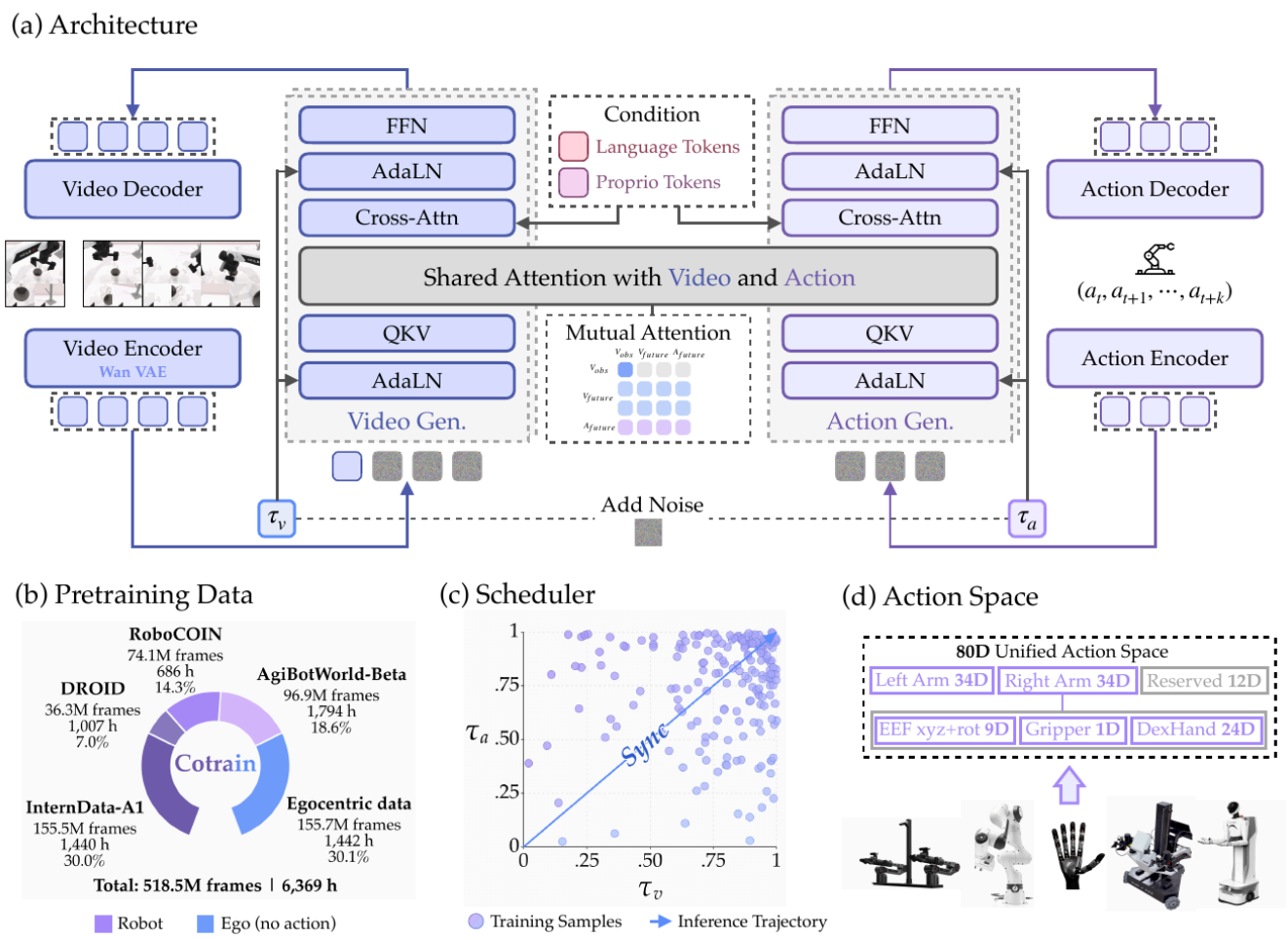
게시용 버전 · 원문 https://arxiv.org/abs/2609.07398 · arXiv:2609.07398v1 · NUS, 칭화대, 베이징대, 홍콩대 외 (Wang, Huang, Li, Zhang, … Lin Shao, Hang Zhao) · 코드 OpenWAM-Official/OpenWAM @ a00c4ba (Apache-2.0) · 분석일 2026-11-04 · 논문 그림 © 저자, CC BY 4.0



논문 Fig. 1: OpenWAM-Infra(모듈형 학습·배포·평가), OpenWAM-Study(통제 실험), OpenWAM-α(518.5M 프레임 사전학습 모델).

핵심 요약

1. **무엇을 했나:** World-Action Model(영상 생성 prior + 행동 생성)의 설계 선택을 부품(인코더, 스트림 백본, 가시성 마스크, denoising 경로, 데이터)으로 쪼개 RoboTwin 2.0에서 같은 조건으로 비교했다. 결론을 모아 Wan2.2-TI2V-5B DiT + 1B ActionDiT를 518.5M 프레임(사람 영상 30%, 실제 로봇 40%, 합성 30%)으로 사전학습했다.
2. **큰 효과는 확실하고 작은 효과는 잡음이다:** 사전학습의 OOD 이득(+9.3-12.1%p), Single → Dual(+6.9%p), 학습 때 영상→행동 경로(+5.0%p)는 z > 10이다. Finding 3의 "mutual 마스크가 일관되게 낫다"(p 0.29-0.67), 공동학습 vs 2단계(p = 0.89), "동기 denoising이 최선"(최선 비동기 대비 p = 0.06)은 seed 1개 설계로 가를 수 없다. 같은 설정으로 보이는 실행도 92.15-93.0으로 흔들린다.
3. **추론 때 미래 영상은 거의 쓰이지 않는다:** 학습 표본은 index 좌표에서 noise 평면 전체에 균등하므로 "비동기 경로가 분포 밖"이라는 설명은 성립하지 않는다. 행동을 순수 noise 영상과 함께 끝내는 경로도 92.3%(동기 93.0%)다. 이상적 denoiser toy에서는 순서가 반대로 나오고, 실제 순서는 행동 스트림의 최대 step 크기와만 상관된다(ρ = −0.82).
4. **표 수치는 맞고 서술 일부가 넘친다:** 부록 D 평균은 모두 재현된다(LIBERO-Plus·RoboCasa365·EBench는 가중 평균, RoboDojo는 5차원 평균, 공개 시드별 파일로 14칸 일치). "최상위권, 근소한 차이"는 RoboCasa365(−19.2%p)에서 맞지 않고, Franka 실물의 LingBot-VA 대비 우위는 p = 0.42다.
5. **코드와 논문의 차이:** 사람 영상(30%) 로더와 데이터가 없어 사전학습 혼합을 만들 수 없다. 사람 영상 표본이 행동 손실 평균에 0으로 들어가 실효 λ_a ≈ 0.70이다. 로더가 fps를 맞추지 않아 같은 32-step chunk가 1.1-3.2초로 달라진다. 평가 설정(velocity 캐시)이 벤치마크마다 다르다.
6. **구현 가능성:** 코드, 설정, 체크포인트 46개(ablation 32개 포함)가 공개되어 평가·SFT·Fig. 9 경로 실험은 CUDA GPU에서 재현할 수 있다. 사전학습(H200 약 21,500 GPU-h, 비공개 사람 영상)은 재현할 수 없다. 이 맥(M4 Pro 24 GiB)은 학습 불가, 공식 경로 추론 불가(체크포인트 24.8 GB, CUDA 전제)이고, 텍스트 인코더를 덜어 낸 MPS 추론(13.4 GB)은 미검증이다.



논문 Fig. 12: OpenWAM-α의 구조, 사전학습 혼합, timestep 표본(학습은 평면 전체, 추론은 대각선), 80D 통합 행동 공간.

01. 논문 분석: OpenWAM

- **논문:** Wang, Huang, Li, Zhang et al. *OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining.* arXiv:2609.07398v1 (2026-09-07). NUS, 칭화대, 베이징대, 홍콩대, 저장대, 홍콩중문대, 상하이교통대. 프로젝트 리드: Yuran Wang, Siqiao Huang, 공동 지도: Lin Shao, Hang Zhao.
- **라이선스:** arXiv abs 페이지 기준 CC BY 4.0. PDF 안에 다른 권리 고지는 없습니다(43쪽 전체 텍스트 확인).
- **공개물:** 코드 `OpenWAM-Official/OpenWAM` (Apache-2.0), Hugging Face 모델 46개(사전학습 1, 벤치마크·실물 SFT 13, Study ablation 32), 데이터셋 9개. 자세한 표는 `02` §4에 있습니다.
- **한 줄 요약:** World-Action Model(WAM, 영상 생성 모델을 이어받아 미래 영상과 행동 chunk를 함께 생성하는 정책)의 설계 선택을 부품으로 쪼개 같은 조건에서 비교하고(OpenWAM-Infra, OpenWAM-Study), 그 결론을 모아 5B 영상 DiT + 1B ActionDiT 모델을 518.5M 프레임으로 사전학습(OpenWAM-α)한 연구입니다.

1. Abstract와 문제 설정

영상 생성 모델은 "세상이 어떻게 변하는가"를 배웠고, 로봇 궤적은 "무엇을 하면 되는가"를 줍니다. WAM은 앞의 것을 초기값으로 삼아 뒤의 것을 배웁니다(§1). 저자들은 기존 WAM 시스템이 백본, 시각 표현, 구조, 정보 흐름, 추론 방식, 데이터가 한 덩어리로 묶여 있어서, 무엇이 성능을 만들었는지 가를 수 없다고 봅니다(§1, §2). 그래서 세 질문을 세웁니다.

질문	다루는 절	답(Finding)
Q1. WAM은 어떤 세계 지식을 물려받아야 하나	§4.1	충분히 큰 생성 백본과 압축되고 정보가 많은 latent (Finding 1)
Q2. 세계 학습과 행동 학습의 시너지는 어떻게 만드나	§4.2	행동 전용 용량, 학습 때 영상→행동 정보 경로, 추론 때 동기 denoising (Finding 2)
Q3. 그 시너지를 여러 도메인으로 키우려면	§4.3	사전학습은 주로 OOD를 늘린다. 사람 영상 + 로봇 데이터 1단계 공동학습, mutual 마스크 (Finding 3)

기여는 세 층입니다. OpenWAM-Infra(모듈형 학습·배포·평가 코드), OpenWAM-Study(통제 실험), OpenWAM-α(사전학습 모델과 벤치마크 8개 + 실물 3종 평가)입니다.

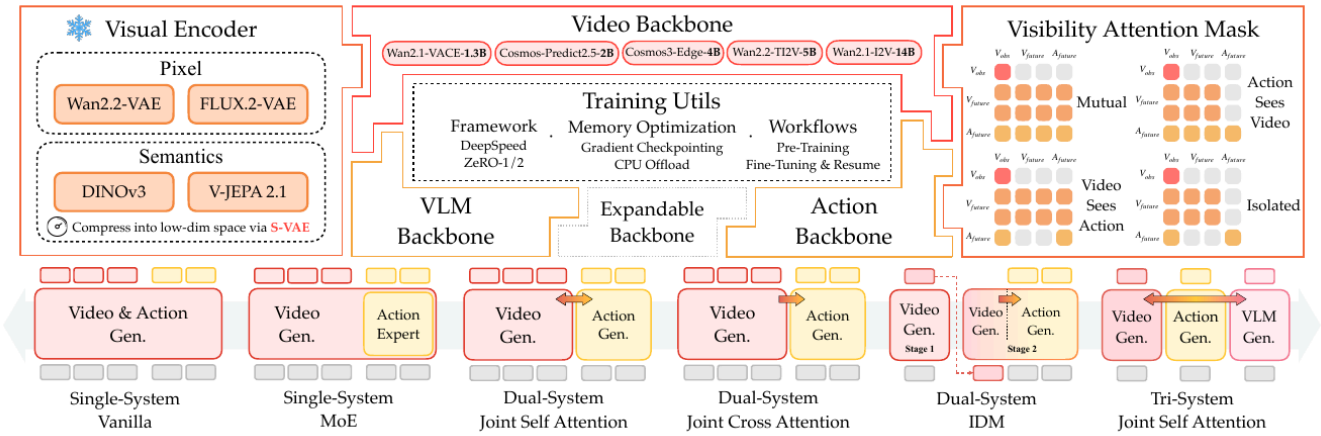
2. Method

2.1 조합형 모델 C(E, S, M) (§3.1)

모델 하나는 세 부품의 조합입니다.

- **시각 인코더 E(항상 고정):** 재구성형 Wan2.2-VAE, FLUX.2-VAE와 표현형 DINOv3, V-JEPA 2.1. 표현형 특징(768-1024차원)은 S-VAE로 48차원으로 줄이는 선택지가 있습니다.
- **스트림 백본 S:** 영상 백본 W(Wan2.1-VACE-1.3B, Cosmos-Predict2.5-2B, Cosmos3-Edge-4B, Wan2.2-TI2V-5B, Wan2.1-I2V-14B), 행동 백본 A(별도 ActionDiT 또는 영상 시퀀스에 합류), 선택적 VLM 백본 U(Qwen3-VL). 모든 백본이 prepare → 층별 block step(주의 전·후 반쪽으로 나눌 수 있음) → finalize라는 같은 계약으로 실행됩니다. 그래서 스트림 사이의 주의를 백본 내부를 건드리지 않고 바꿀 수 있습니다.
- **가시성 마스크 M (Eq. 1):** 2×2 블록 [M_V←V, M_V←A; M_A←V, M_A←A]. 영상 내부는 first-frame causal(깨끗한 첫 프레임은 자기만 봄), 행동 내부는 양방향으로 고정하고, 교차 블록 두 개로 네 모드를 만듭니다: mutual, action-sees-video, video-sees-action, isolated.

구조는 세 계열 일곱 변형입니다(Fig. 2 아래 줄, Table 1). Single-System(Vanilla, 행동 토큰만 다른 FFN으로 보내는 MoE), Dual-System(joint self-attention, joint cross-attention과 그 gradient 차단판, 영상을 먼저 만들고 행동을 역동역학으로 푸는 IDM), Tri-System(+Qwen3-VL, 읽기 전용 꼬리)입니다. 대표 시스템으로 Single은 Cosmos Policy·DreamZero, Dual은 Fast-WAM·LingBot-VA, Tri는 Motus를 듭니다.



논문 Fig. 2: 위는 교체 가능한 부품 세 종류(시각 인코더, 스트림 백본, 가시성 마스크), 아래는 구조 7종.

2.2 학습: 결합 flow matching (§3.2, §5.1.2)

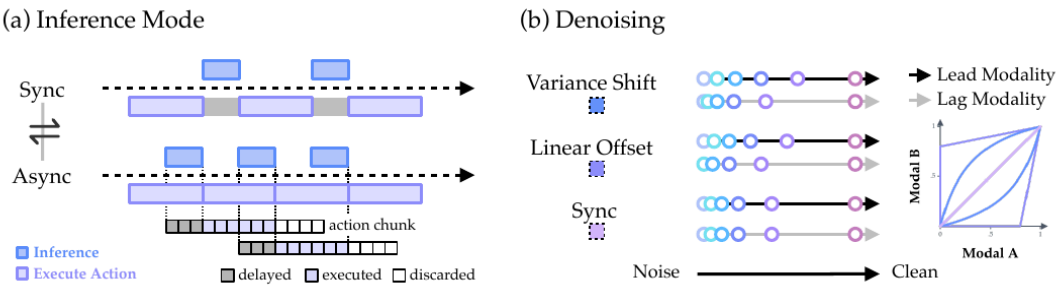
표본은 (지시문 ℓ, 영상 창 $o_{1:T}$, 행동 chunk $a_{1:H}$, 고유수용 상태 q, 차원별 유효 마스크 m)입니다. 논문 표기로 t = 0이 순수 noise, t = 1이 깨끗한 데이터이고, 두 스트림이 **각자 독립으로 뽑은** t_v, t_a로 noise를 받습니다.

$$L = \lambda_v \, E[w(t_v) \, \| \hat{v}_z - (z - \varepsilon_v) \|^2] + \lambda_a \, E[w(t_a) \, \| m \odot (\hat{v}_a - (a - \varepsilon_a)) \|^2] \quad \text{(Eq. 2)}$$

깨끗한 조건 프레임은 영상 손실에서 빠지고, m이 각 embodiment가 실제로 채운 행동 좌표만 남깁니다. 저자들은 t_v, t_a를 독립으로 뽑으므로 학습이 (t_v, t_a) 평면 전체를 덮고, 따라서 어떤 추론 경로도 분포 안이라고 주장합니다(§3.2). OpenWAM-α는 $\lambda_v = \lambda_a = 1$, 중간 noise에서 최대인 종 모양 $w(\cdot)$, $t = 1 - f_\rho(u)$(u 균등, $\rho = 5$로 고noise 쪽에 치우침)를 씁니다(§5.1.2). 실제 식과 학습 분포의 모양은 `03` §1, 코드 대조는 `04` D4-D6에 있습니다.

2.3 배포: 추론 모드, denoising 경로, 가속 (§3.3)

- **추론 모드:** 동기(버퍼가 비면 멈추고 새 chunk를 추론)와 비동기(남은 행동이 d개일 때 다음 chunk를 미리 추론하고, 앞 d개는 건너뛰고 n개를 실행, 나머지는 버림).
- **denoising 경로 (Eq. 3):** 결합 noise 평면 위의 경로 $\tau = \{(t_v^i, t_a^i)\}$. 동기는 대각선입니다. 비동기는 분산 이동 $f_\alpha(s) = \alpha s / (1 + (\alpha - 1)s)$로 선행 스트림을 먼저 깨끗하게 하거나, 선형 지연 $h_o(s) = \max\{(s - o)/(1 - o), 0\}$로 후행 스트림을 o까지 순수 noise에 둡니다. $(\alpha, o) = (1, 0)$이 동기와 정확히 같습니다.
- **가속:** prompt 임베딩 캐시, 영상 decode 생략, torch.compile + CUDA graph, DiT velocity 캐시(최근 두 스트림 velocity의 cosine 유사도가 임계값 이상이면 다음 forward를 건너뛰고 캐시 값으로 적분). RTX 5090에서 Dual JSA 175 ms, 기준 대비 2.45배 빠릅니다(Fig. 4).



논문 Fig. 3: (a) 동기·비동기 추론 모드 (b) 분산 이동, 선형 지연, 동기 denoising 경로.

2.4 평가 규약과 80차원 통합 행동 공간 (§3.4)

벤치마크는 WebSocket 클라이언트로 정책 서버에 붙고, 관측(카메라 최대 3개, 지시문, 상태)을 보내 행동 한 개를 받습니다. 정규화, 카메라 배치, 역정규화는 서버가 체크포인트 기록대로 처리합니다. 실물 로봇도 같은 인터페이스를 씁니다. 여러 embodiment를 한 모델로 학습할 때는 **80차원 통합 행동 공간**을 씁니다. 팔 블록 34차원 × 2(말단 위치 3 + 6D 회전 6 + 그리퍼 1 + 손 24)와 예비 12칸이고, 각 데이터셋이 index map π로 흩어 넣습니다(Eq. 4). 채워지지 않은 칸은 m으로 손실에서 빠집니다.

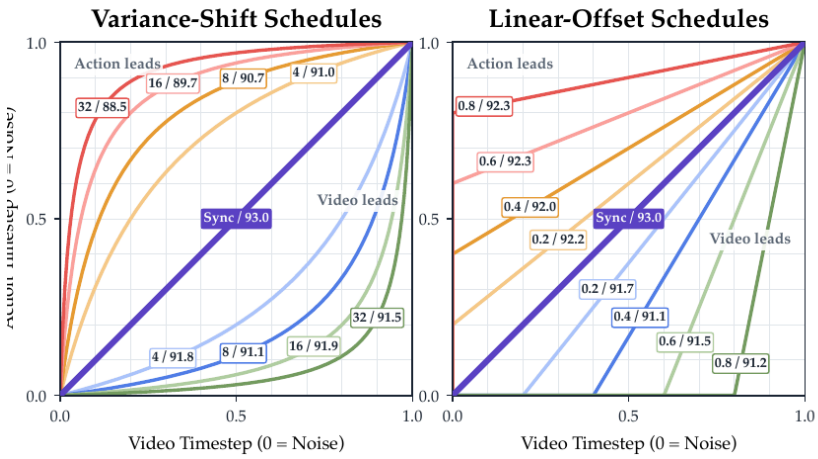
3. Experiments

3.1 OpenWAM-Study (§4): RoboTwin 2.0에서의 통제 실험

평가는 RoboTwin 2.0 두 설정입니다. **Full**(clean 2,500 + randomized 25,000 시연으로 학습, 두 조건 평가)과 **Clean2Random**(clean만 학습, randomized가 OOD)입니다.

실험	결과	출처
영상 백본 크기	1.3B 90.14 → 2B 91.64 → 5B 92.39 → 14B 93.79. 5B를 기본으로 채택	Fig. 6
시각 인코더 (DiT 무작위 초기화)	DINOv3 76.42, V-JEPA 80.91, FLUX.2 84.26, V-JEPA+S-VAE 88.46, DINOv3+S-VAE 90.18, Wan2.2-VAE 90.30	Fig. 7
구조	Single Vanilla 85.50, MoE 84.63, Dual JSA 92.36, JCA 88.25, Detached CA 91.85, IDM 87.95, Tri JSA 92.60	Table 1
학습 때 마스크	Isolated 87.41, Video Sees Action 87.63, Action Sees Video 92.39, Mutual 92.15	Table 2
추론 때 경로	동기 93.0. 비동기 16개는 88.5-92.3으로 모두 낮음	Fig. 9
사전학습 (600시간 예산)	OOD: scratch 14.50 → robot only 23.80 → 2단계 26.50 → 공동학습 26.62. ID는 87.00-88.50	Fig. 10
사전학습 후 마스크	Mutual – ASV: scratch Full –0.24, 사전학습 Full +0.16, C2R ID +0.70, OOD +0.72	Fig. 11

저자들의 결론(Table 3)은 Wan2.2-TI2V-5B + 압축 latent, Dual joint self-attention + 동기 denoising, 사람 영상 + 로봇 1단계 공동학습 + mutual 마스크입니다.



논문 Fig. 9: 비동기 경로 16개와 동기의 RoboTwin2.0-Full 성공률.

해석 (재계산은 `03` §2-3, `04` C5-C6):

- 큰 효과는 확실합니다. Single → Dual(+6.9%p), 영상→행동 경로 유무(+5.0%p), 사전학습의 OOD 이득(+9.3-12.1%p)은 이항 표본 오차의 10배가 넘습니다.
- 그 밖의 차이는 대부분 잡음 범위입니다. RoboTwin 평가는 과제 50개 × 100 episode, seed 0 한 번이라 평균의 SE가 약 0.27%p, 두 실행 차이의 SE가 약 0.37%p입니다. Tri vs Dual(+0.24), Mutual vs ASV(Fig. 11의 +0.16 / +0.70 / +0.72), 공동학습 vs 2단계(+0.12)는 p > 0.25입니다. 그런데도 Finding 3은 "mutual이 일관되게 낫다"고 씁니다.
- 같은 설정으로 보이는 실행 사이의 차이가 이 효과들보다 큽니다. Dual JSA는 Table 1에서 92.36, Table 2 mutual 92.15, Table 2 ASV 92.39, Fig. 9 동기(mutual, 독립 noise) 93.0입니다.

3.2 OpenWAM-α (§5)

- **구조 (§5.1.1, Fig. 12a):** 고정 Wan2.2-VAE + Wan2.2-TI2V-5B DiT(폭 3072, 30층) + 1B ActionDiT(폭 1024). 30층 모두 bridge이고, 공유 주의 공간은 24 head × 128차원, mutual 마스크입니다. 첫 latent 프레임은 현재 관측이고 t_v = 1로 고정됩니다. umT5 4096차원 문맥에 q를 토큰 하나로 붙여 두 스트림이 각자 cross-attention으로 읽습니다.
- **학습 (§5.1.2, Table 9):** H200 128장 × 약 7일, global batch 3,072, 155,862 iteration(1 epoch), lr 1e-4 cosine(warmup 5%), AdamW (0.9, 0.95), ZeRO-2, 384 × 320, 33프레임 창(영상 stride 4), 흐름 shift 5/5.
- **데이터 (§5.2, Table 4):** 원 데이터 1.33B 프레임(14,348시간) → 정제·표본추출 후 518.5M 프레임(6,369시간). 자체 1인칭 사람 영상 30.1%(공개 안 됨), AgiBotWorld-Beta 18.6%, RoboCOIN 14.3%, DROID 7.0%, 합성 InternData-A1 30.0%. 사람 영상은 행동이 없어 영상 스트림만 학습시킵니다. 정제는 시각 품질 검사와 로봇 신호 4단계(신호 무결성, 상태 기반 정지 구간 자르기, 시각 교차 확인, 에피소드 삭제)입니다(§5.2.2).
- **배포 (§5.1.3):** 동기 경로, N = 10, 동기 추론 모드, RTX 5090에서 chunk당 약 170 ms입니다. 저자들은 동기 모드를 쓰므로 "결과가 모델 자체의 성능을 가장 직접 반영한다"고 씁니다. 하지만 velocity 캐시(근사)를 켠 벤치마크가 있습니다(`04` D3).

3.3 시뮬레이션 벤치마크 8개 (§5.3, Table 5, Table 13-20, Fig. 13-16)

벤치마크	OpenWAM-α	표의 1위	순위
LIBERO (Table 13)	99.3	ABot-M0.5 99.4	2/18
VLABench SR (Table 14)	58.9	Xiaomi-Robotics-1 59.1	2/6 (IS 기준 5/6)
RoboTwin2.0-Full (Table 16)	93.60	ABot-M0.5 94.10	4/14 (LingBot-VA 2.0과 동률)
RoboTwin2.0-Clean2Random (Table 15)	69.0	Qwen-RobotManip 77.1	2/13, WAM 중 1위(다음 4D-WAM 61.7)

벤치마크	OpenWAM-α	표의 1위	순위
RoboDojo SR (Table 17)	11.92	DM0.5 19.34	4/13, WAM 중 1위(다음 X-WAM 3.83)
EBench SR / Score (Table 18)	49.4 / 64.7	(1위)	1/6
RoboCasa365 (Table 19)	38.2	Xiaomi-Robotics-1 57.4	3/10
RoboCasa-GR1 (Table 20)	60.5	PhysBrain 1.0 64.5	2/15
LIBERO-Plus (Table 5)	69.2	Qwen-RobotManip 89.0	11/13

- 저자들은 LIBERO, VLABench, RoboTwin-Full, RoboCasa365, RoboCasa-GR1에서 "1위와 근소한 차이의 최상위권"이라고 씁니다(§5.3.1). RoboCasa365는 1위와 19.2%p 차이라서 이 서술에 맞지 않습니다(`04` C7).
- LIBERO-Plus의 낮은 점수는 카메라(33.8)와 noise(39.8) 교란에 몰려 있습니다. 저자들은 원인을 두 가지로 봅니다. 하나는 단일 팔 사전학습 데이터 부족(Fig. 15)이고, 다른 하나는 픽셀 latent 미래 예측의 시각 교란 취약성입니다(Takeaway 1, 2). LIBERO-Plus는 LIBERO 체크포인트를 추가 미세조정 없이 평가했습니다(B.2).
- VLA 대 WAM(§5.3.2, Fig. 16): 분포 안에서는 WAM이 더 잘 맞추고 분포 밖에서는 VLA가 더 잘 일반화하며, 둘 다 데이터로 보완된다는 것이 Takeaway 3입니다. 통제된 비교는 StarVLA 대 Fast-WAM 한 쌍뿐이고, 나머지는 데이터·구조·학습이 모두 다른 모델끼리의 비교입니다.

3.4 실물 로봇 (§5.4, Table 6-8, 부록 C)

플랫폼	비교	결과	통계 (`04` C9)
Franka 단일 팔 6과제 × 20회 (Table 6)	π0.5, LingBot-VA	99/120 vs 93/120 vs 66/120	LingBot-VA와 차이 p = 0.42 (유의하지 않음), π0.5와는 $p = 7\times10^{-6}$
RoboDojo-Real 3 embodiment × 6과제 × 10회 (Table 7)	VLA 5종	Score 37.6 / SR 24.4 vs π0.5 22.9 / 12.8	SR 44/180 vs 23/180, p = 0.006. Score 기준 18과제 중 13과제 최고
Wuji 손 + Tianji 팔 4과제 (Table 8)	π0.5	ID SR 27/40 vs 12/40, OOD 40/65 vs 19/65	합치면 p = 0.002, 0.0004. 과제별로는 8개 중 2개만 p < 0.05

Wuji 손의 9D 말단 자세 + 21자유도 손은 사전학습에 없던 embodiment입니다. Collect Shuttlecocks OOD는 점수 분모가 π0.5 54, OpenWAM-α 57로 달라서 시행 조건이 같지 않습니다.

4. Limitations

저자들이 적은 한계(부록 A)는 사후학습(post-training)을 다루지 않았다는 점, 융합 방식이 교차 주의와 hard-routed MoE에 머물렀다는 점, UMI식 데이터가 빠졌다는 점, Wan2.2-VAE가 시점·잡음·장면 변화에 약하다는 점입니다.

비판적으로 읽으면 다음이 더 중요합니다.

1. **"체계적"의 통계적 기반이 얇습니다.** 통제 실험은 모두 RoboTwin 2.0 한 벤치마크, 학습 seed 1개, 평가 seed 1개입니다. Finding 2의 "동기가 최선"(최선 비동기 대비 +0.7%p, p = 0.06)과 Finding 3의 "mutual이 일관되게 낫다"(p 0.29-0.67)는 이 설계로는 가를 수 없습니다(`03` §3).
2. **추론 때 미래 영상의 기여가 작습니다.** 행동 선행 선형 지연 o = 0.8은 행동을 거의 순수 noise인 영상($\sigma_v \geq 0.83$)과 함께 끝내는데도 92.3%로, 동기 93.0%와 0.7%p 차이입니다. 추론 때 생성된 미래가 거의 쓰이지 않는다는 뜻이고, 저자들이 지지한다고 쓴 "표현 학습 가설"과 맞습니다. 반면 "추론 때 미래 예측이 행동을 이끈다"는 WAM의 일반적 홍보와는 다릅니다(`03` §2).
3. **사전학습 재현의 핵심 데이터가 비공개입니다.** 전체 프레임의 30%를 차지하는 1인칭 사람 영상(WorldEngine)과 그 로더가 공개되지 않았습니다. Study의 EgoDex 로더도 없습니다(`04` D1).

4. **평가 설정이 벤치마크마다 다릅니다.** RoboTwin은 velocity 캐시를 켰고 EBench는 껐습니다. 공식 README는 캐시를 끄면 RoboTwin 수치가 바뀐다고 적었습니다(`04` D3).
5. **Takeaway 3(VLA vs WAM)은 상관 관찰입니다.** 비교 모델 대부분이 데이터·구조·학습이 모두 달라서 패러다임 효과로 분리되지 않습니다.
6. **하드웨어 장벽이 큽니다.** 사전학습은 H200 약 21,500 GPU-h, 체크포인트 하나가 24.8 GB입니다(`04` §4).

03. 유도와 재계산: noise 평면, 효과 크기, 데이터·연산

재계산 스크립트는 `analysis/table_check.py` (→ `table_check_results.json`)와 `analysis/noise_toy.py` (→ `noise_toy_results.json`)입니다. 둘 다 단일 스레드 numpy로 이 맥에서 각각 약 1초, 약 1분 걸립니다.

1. 결합 noise 평면을 코드 식으로 다시 쓰기

1.1 부호와 좌표

논문은 t = 0이 noise, t = 1이 데이터이고 목표 velocity가 z − ε입니다(Eq. 2). 코드는 σ = 1이 noise이고 $x_\sigma = (1-\sigma)x + \sigma\varepsilon$, 목표가 ε − x입니다(`action_backbone/scheduler.py`). σ = 1 − t로 바꾸면 dx/dσ = −dx/dt라서 두 식은 같습니다.

학습 grid는 u = k/1000(k = 0…999)을 $\sigma = f_5(1-u)$로 보낸 1000칸입니다. $f_\rho(s) = \rho s / (1 + (\rho - 1)s)$입니다. 두 스트림은 index k를 **각자 독립으로 균등하게** 뽑습니다(`architectures/base.py` 1089, 1113행). 따라서

> index 좌표 (u_v, u_a)에서 학습 표본은 단위 정사각형 위에 균등 분포합니다.

σ 좌표로 보면 shift 5 때문에 noise 쪽이 25배 조밀합니다. 표본의 35.8%가 σ ≥ 0.9이고, σ < 0.1은 2.1%뿐입니다.

1.2 손실 가중치

"종 모양 가중치"(§5.1.2)의 실제 식은 다음과 같습니다(`_set_training_weight`).

$w(\sigma) \propto \exp(-2(\sigma - 0.5)^2) - \min_{\text{grid}} \exp(-2(\sigma - 0.5)^2)$

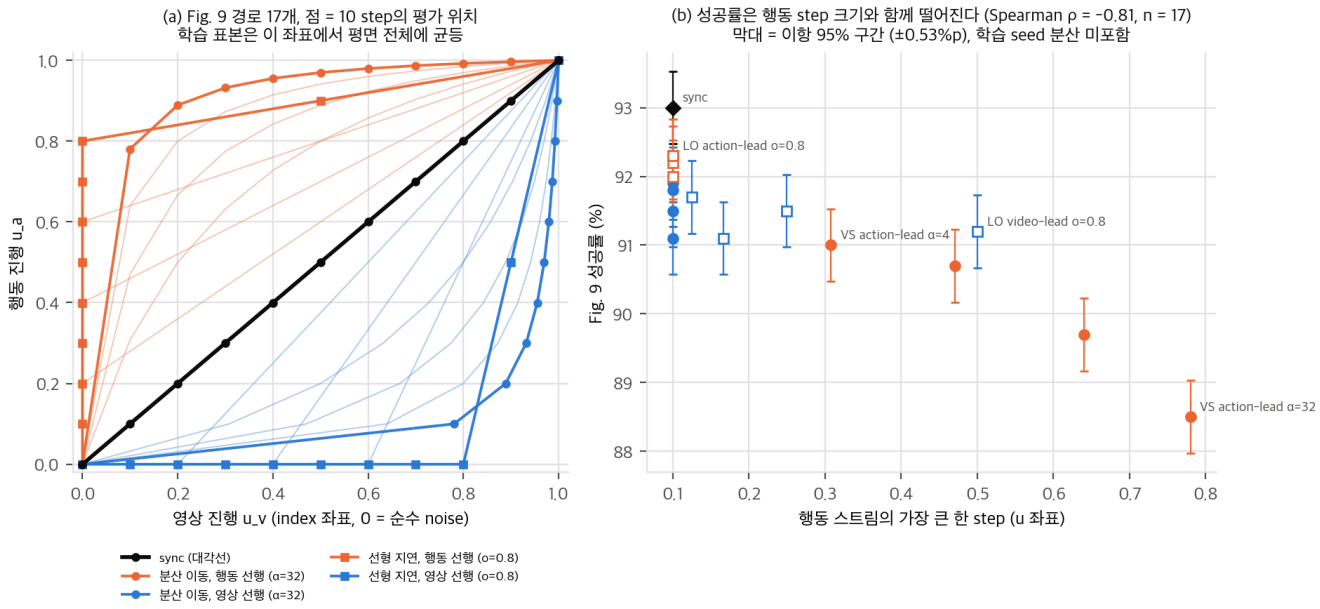
최솟값이 σ = 1에서 나오므로 **순수 noise에서 가중치가 정확히 0**이고, σ = 0.5에서 최대, σ → 0에서도 거의 0입니다. 표본 밀도와 곱하면 학습 신호의 65%가 $\sigma \in [0.5, 0.9]$에 몰립니다. σ ≥ 0.9는 표본의 36%지만 가중 손실의 10%입니다(`noise_toy_results.json` A).

1.3 비동기 경로의 식

Eq. 3의 선행 곡선은 진행도 s(0 = noise)에서 깨끗함 $f_\alpha(s)$입니다. 코드는 σ 좌표 s' = 1 − s에서 계산하는데, 다음 항등식을 씁니다.

$1 - f_\alpha(1-s') = 1 - \alpha(1-s') / (1 + (\alpha - 1)(1-s')) = s' / (\alpha - (\alpha-1)s') = f_{1/\alpha}(s')$

그래서 `lead_sigma = f_{1/α}(s')`이고 α = 1에서 비트 단위로 동기와 같습니다. 선형 지연 h_o는 σ 좌표에서 $\min(s'/(1-o), 1)$입니다. 두 경로 모두 마지막에 각 스트림의 shift 5 grid를 통과합니다(`deploy/denoise_schedule.py`). 결과적으로 Fig. 9의 경로 17개는 모두 index 좌표의 단위 정사각형 안에 있고, 학습 표본 밀도는 어느 경로에서나 같습니다.



우리 분석: (a) Fig. 9 경로 17개를 코드 식 그대로 index 좌표에 그림(점 = 10 step의 평가 위치). (b) Fig. 9 성공률 대 행동 스트림의 가장 큰 한 step.

2. Fig. 9: 비동기 경로는 왜 나쁜가

2.1 "분포 밖이라서"는 아닙니다

1.1에 따라 어떤 경로도 학습 표본 밀도가 같습니다. 대각선 근처 띠($|u_v - u_a| < 0.05$)의 질량은 9.75%로, 다른 경로 근처 띠와 다르지 않습니다. 논문의 "모든 경로가 분포 안" 서술(§3.2)은 표본 밀도로는 정확히 맞습니다. 따라서 비동기 경로가 나쁜 이유를 학습 표본 부족으로 설명할 수 없습니다.

2.2 이상적 denoiser toy: 경로 자체의 이산화 오차

`noise_toy.py` B는 (미래 영상 8차원, 행동 4차원)이 4개 봉우리의 Gaussian 혼합인 세계를 만듭니다. 영상 평균은 행동 평균의 선형 사상이라 영상이 어느 봉우리인지 알려 줍니다. 정확한 사후 평균으로 velocity를 만들어 각 경로를 10 step Euler로 적분하고, 최종 행동이 어떤 봉우리의 99% 영역 안에 들어간 비율("유효 행동")을 셉니다.

경로 (봉우리 폭 0.15)	유효 행동	Fig. 9
동기	0.949	93.0
분산 이동, 영상 선행 α = 4-32	0.985-0.994	91.1-91.9
분산 이동, 행동 선행 α = 4-32	0.967-0.986	88.5-91.0
선형 지연, 영상 선행 o = 0.2-0.8	0.917-0.942	91.1-91.7
선형 지연, 행동 선행 o = 0.2-0.8	0.832-0.927	92.0-92.3
동기 400 step (기준)	0.993	-

- 모델이 완벽하면 400 step에서 모든 경로가 같은 분포에 도달합니다(동기 0.993, 행동 선행 α = 32도 0.990). 경로 차이는 10 step 이산화에서만 생깁니다.
- 이상적 모델에서는 **영상 선행이 동기보다 낫고, 행동 선행 선형 지연이 가장 나쁩니다.** 영상이 먼저 봉우리를 정해 주면 행동이 한 봉우리 안의 단봉 문제가 되기 때문입니다. Fig. 9와의 Spearman 상관은 −0.45(봉우리 폭 0.10, 0.25에서도 −0.48, −0.45)로 **순서가 반대**입니다.

- 결론: Fig. 9의 순서는 경로의 수학적 성질에서 나오지 않습니다. 학습된 네트워크가 평면의 어느 영역을 어떻게 근사하는지에서 나옵니다.

2.3 실제 순서를 따르는 단 하나의 경로 통계

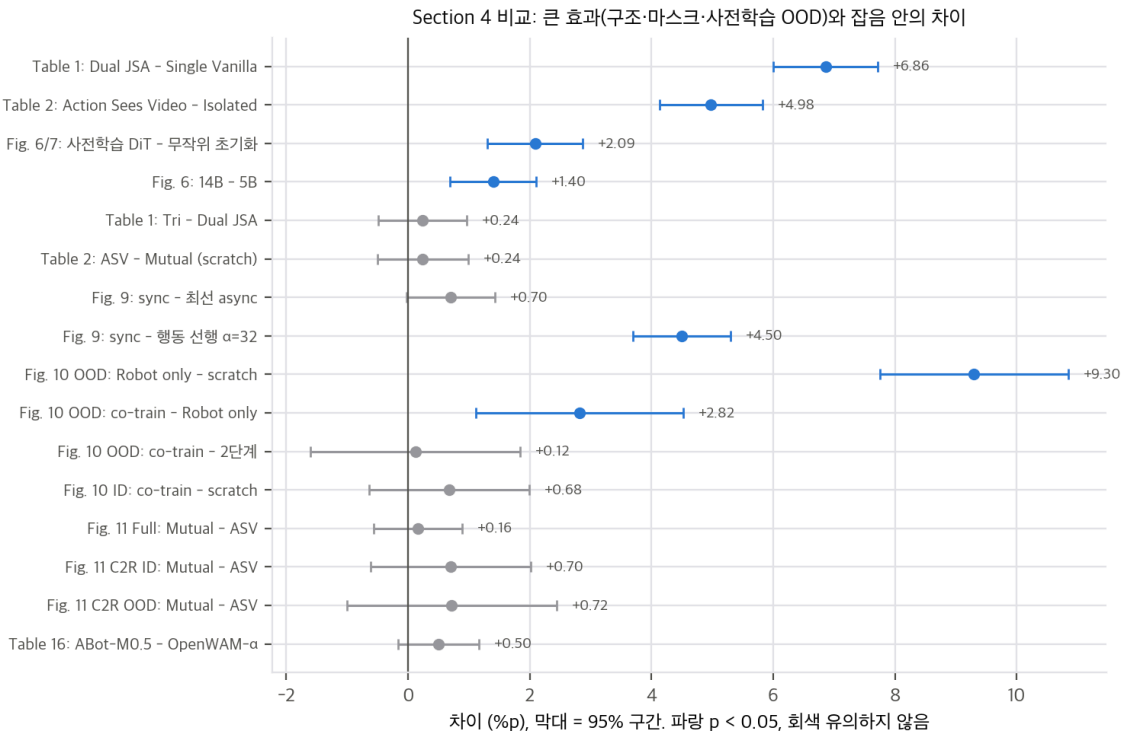
경로별 통계 다섯 개 가운데 Fig. 9 성공률과 강하게 상관된 것은 **행동 스트림이 한 번에 건너뛰는 index 거리의 최댓값**뿐입니다 (Spearman $\rho = -0.82$, $p = 7\times10^{-5}$, $n = 17$). 행동 선행 α가 커질수록 첫 step이 $\sigma_a = 1 \rightarrow 0.58$($\alpha = 32$, u 거리 0.78)로 커지고, 성공률이 91.0 → 88.5로 떨어집니다. 행동이 갱신되는 횟수($\rho = 0.15$), 행동이 절반쯤 정해질 때 영상 noise($\rho = -0.23$), 손실 가중치 평균($\rho = -0.03$)은 상관이 없습니다.

해석에는 주의가 필요합니다. 값이 0.1 단위로 반올림되어 있고, 16개 비동기 값 대부분이 서로 표본 오차(±0.53%p, 95%) 안이며, 상관은 사실상 행동 선행 분산 이동 계열 4점이 만듭니다. 그래도 다음 두 가지는 말할 수 있습니다.

1. **학습된 행동 denoiser는 큰 step에 약합니다.** 이상적 denoiser에서는 큰 step이 문제가 되지 않았으므로, 이것은 모델 오차의 성질입니다. N = 10을 유지하면서 경로를 바꾸려면 행동 스트림의 step 간격을 균등하게 두는 것이 안전합니다.
2. **추론 때 생성된 미래 영상은 거의 쓰이지 않습니다.** 행동 선행 선형 지연 $o = 0.8$은 행동 10 step 동안 영상이 $\sigma_v = 1$(9번), 0.83(1번)입니다. 즉 깨끗한 현재 프레임만 보고 행동을 끝내는데도 92.3%입니다. 동기 93.0%와의 차이 0.7%p는 $p = 0.06$입니다. 영상→행동 정보 경로가 학습 때는 필수(Table 2, +5.0%p)지만 추론 때는 기여가 1%p 안팎이라는 뜻입니다. 저자들의 "표현 학습 가설 지지"(§4.2.3)와 같은 결론이고, 이 경로 실험이 그보다 더 직접적인 증거입니다. 실용적으로는 추론 때 영상 스트림을 얕게 돌리거나 생략하는 가속 여지가 있습니다.

3. 무엇이 성능을 만드는가: 효과 크기와 잡음

RoboTwin 평가는 과제 50개 × 100 episode, seed 0 한 번입니다(공식 `benchmarks/robotwin/README.md`). 조건 하나는 N = 5,000, 두 조건 평균은 N = 10,000입니다. 92% 근처에서 평균 SE는 0.27%p, 두 실행 차이의 SE는 약 0.37%p입니다. 과제가 고정되어 있으므로 이 이항 SE는 episode 표본 오차의 상한입니다. 그러나 학습 seed 분산이 빠져 있어 실제 불확실성은 이보다 큽니다.



우리 분석: Section 4 비교의 차이와 95% 구간. 파랑은 $p < 0.05$.

서열	효과	차이	z
1	사전학습의 OOD 이득 (Fig. 10, robot only – scratch)	+9.3%p	11.8
2	구조 용량 (Table 1, Dual JSA – Single Vanilla)	+6.9%p	15.5
3	학습 때 영상→행동 경로 (Table 2, ASV – Isolated)	+5.0%p	11.7
4	추론 경로의 큰 행동 step (Fig. 9, 동기 – 행동 선행 α = 32)	+4.5%p	11.0
5	사람 영상 추가의 OOD 이득 (Fig. 10, 공동학습 – robot only)	+2.8%p	3.3
6	사전학습된 DiT 초기화 (Fig. 6 5B 92.39 – Fig. 7 무작위 초기화 90.30)	+2.1%p	5.3
7	백본 5B → 14B (Fig. 6)	+1.4%p	3.9
-	Tri – Dual, ASV – Mutual(scratch), 공동학습 – 2단계, Mutual – ASV(사전학습 후 세 경우), 동기 – 최선 비동기	0.1-0.7%p	0.1-1.9

여기서 보이는 것은 다음과 같습니다.

- **Finding 3의 "mutual이 일관되게 낫다"는 지지되지 않습니다.** 세 비교 모두 p > 0.29입니다. 방향이 세 번 같을 확률(1/8)을 쳐도 약합니다.
- **같은 설정의 실행 사이 차이가 이 효과들과 비슷합니다.** Dual JSA mutual 계열 값이 92.15(Table 2), 92.36(Table 1), 93.0(Fig. 9)입니다. 93.0 vs 92.15는 z = 2.3입니다. 학습 seed 분산 또는 보고되지 않은 설정 차이가 0.5-1%p 수준이라는 뜻입니다.
- **사전학습된 영상 생성 prior의 몫은 생각보다 작습니다.** 같은 5B 구조를 무작위로 초기화해도 90.30이고, 사전학습 초기화는 92.39입니다(+2.1%p). 무작위 초기화 5B는 사전학습된 Wan2.1-VACE-1.3B(90.14)와 같습니다. RoboTwin-Full처럼 학습 데이터가 많은 ID 설정에서는 "영상 생성 사전학습이 핵심"이라는 WAM의 전제가 약하게만 확인됩니다. OOD에서 같은 비교는 없습니다.
- **수치 불일치 하나:** Fig. 11의 사전학습 C2R OOD Mutual은 25.70 + 0.72 = 26.42인데, 같은 설정이어야 할 Fig. 10 공동학습 OOD는 26.62입니다(ID는 87.68로 일치).

4. 데이터와 연산 산술

4.1 Table 4

- 프레임 ÷ fps ÷ 3600이 표의 시간과 1시간 안에서 맞습니다. 예외는 RoboCOIN Full로, 104.5M / 30 fps = 967.6시간인데 표는 956시간입니다(1.2% 차이. 일부 하위 데이터셋의 fps가 30이 아닐 수 있습니다).
- 30 / 40 / 30 배분이 정확합니다. 사람 영상 30.03%, 실제 로봇 39.98%, 합성 29.99%입니다. 실제 로봇 40% 안의 배분(AgiBot 46.7%, RoboCOIN 35.8%, DROID 17.5%)은 정제 프레임 비율 그대로입니다(§5.2.1 서술과 일치).
- 공개 사전학습 config의 `total_hours`는 사람 영상과 InternData-A1이 둘 다 1439.0268시간으로 완전히 같습니다. 둘 다 30% 프레임 목표이고 30 fps라서 같은 시간이 됩니다. Table 4는 1,442와 1,440으로 적었고, 실제로 뽑힌 양이 목표보다 조금 많았던 것으로 보입니다.

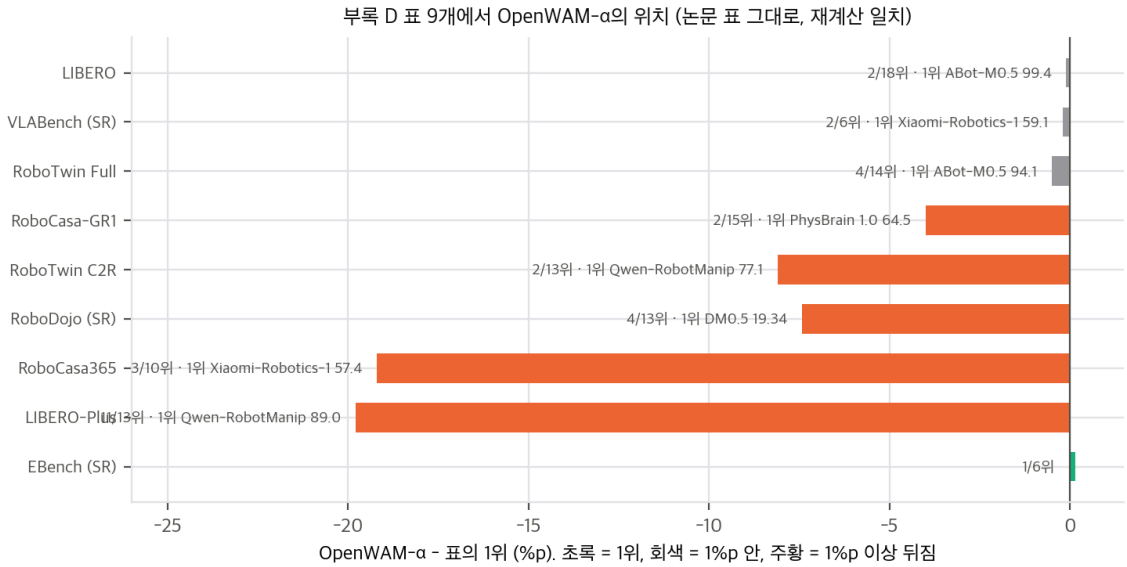
4.2 Table 9와 학습 신호

- 155,862 iteration × 3,072 = **478.8M 클립**입니다. 창 stride가 1이고 코드상 창 수 ≈ 프레임 수이므로(에피소드 끝은 마지막 프레임 반복으로 채움), 1 epoch가 518.5M 프레임보다 7.7% 적은 이유는 설명되지 않습니다.
- 128장 × 7일 = **21,504 H200-hour**, step당 3.88초, GPU당 초당 6.2클립입니다.
- 토큰 수: 33프레임을 stride 4로 9장 → Wan2.2-VAE(시간 4배, 공간 16배) → latent 3프레임 → (1, 2, 2) patch → 12 × 10 = 120 토큰/프레임입니다. 영상 360토큰, 행동 32토큰입니다. DiT·ActionDiT 행렬곱만 세면 6 × (5B × 360 + 1B × 32) ×

478.8M = 5.3×10^{21} FLOPs이고, H200 bf16 dense 989 TFLOPS 기준 MFU 약 7%입니다. 주의 연산, 매 step의 umT5 인코딩, VAE 인코딩을 빼고 센 하한입니다. 데이터 로딩과 실시간 인코딩이 비용의 상당 부분일 가능성이 큽니다.

- **행동 손실의 실효 가중치는 0.7입니다.** 사람 영상 표본은 행동 유효 칸이 없어 표본별 행동 손실이 0이지만, 배치 평균의 분모에는 들어갑니다(`base.py` 1296-1297행). 표본의 30%가 사람 영상이므로 사전학습의 실효 $\lambda_a \approx 0.70$이고, 논문의 $\lambda_a = 1$(Table 9)과 다릅니다. SFT에서는 1입니다.
- SFT 비용도 작지 않습니다. RoboTwin-Full SFT는 118,655 step × 256 = 30.4M 클립으로, 사전학습의 6.3%입니다. RoboCasa365는 61.4M 클립(12.8%)입니다.

5. 벤치마크 표 읽기



우리 분석: 부록 D 표 9개에서 OpenWAM-α와 1위의 차이(논문 표 수치).

- **평균 행의 정의가 표마다 다릅니다.** LIBERO, RoboTwin, VLABench는 단순 평균입니다. RoboCasa365는 과제 수 가중(18 / 16 / 16, RMSE 0.03)이고, EBench는 과제 수 7 / 10 / 9 가중입니다. LIBERO-Plus도 가중 평균입니다(합 1 비음수 최소제곱 RMSE 0.49, 단순 평균 RMSE 1.52). OpenWAM-α의 LIBERO-Plus 단순 평균은 71.3, 가중 평균은 69.2입니다. RoboDojo Avg 11.92는 화면의 6열 평균(12.41)이 아니라, Gen-Std와 Gen-Rand를 합친 일반화 차원과 나머지 4차원을 합친 **5차원 평균**입니다. 표의 13행 모두 이 정의로 0.07 안에서 재현됩니다.
- **RoboDojo는 시드 3개 결과가 공개되어 있습니다**(`assets/robodojo_verification/`). 시드별 평균 SR은 11.80, 11.68, 12.28, SD 0.32입니다. 1위 DM0.5(19.34)와의 차이는 시드 SD의 23배라서 순위는 확실합니다. 42과제 가운데 14과제는 세 시드 모두 성공 0입니다.
- **EBench 기준선의 Score는 정수로 반올림된 리더보드 값입니다.** 계열별 값을 7 / 10 / 9로 가중하면 Qwen-RobotManip Score가 60.96이 되어, 표의 60.0과 다릅니다. "Score +4.7"은 표 값끼리의 차이이고, 계열 값으로 다시 계산하면 +3.7입니다. SR 차이(+3.8)는 재현됩니다.
- **"최상위권, 1위와 근소한 차이"(§5.3.1)는 다섯 벤치마크 중 셋에서만 맞습니다.** LIBERO −0.1, VLABench SR −0.2, RoboTwin-Full −0.5(z = 1.5, 동률 범위)는 맞고, RoboCasa-GR1 −4.0은 2위지만 근소하다고 보기 어렵습니다. RoboCasa365 −19.2는 맞지 않습니다. VLABench도 IS(intention score) 기준으로는 6개 중 5위입니다.

6. 실물 결과의 통계

- Franka(Table 6): 99/120 vs LingBot-VA 93/120은 Fisher p = 0.42입니다. "대부분 과제에서 확실히 앞선다"고 썼지만 6과제 중 4승 1무 1패이고, 차이는 유의하지 않습니다. π0.5(66/120)와의 차이는 확실합니다.

- RoboDojo-Real(Table 7): Embodiment 평균과 전체 평균이 재계산과 일치합니다. 과제당 10회라 과제별 SR 차이는 대부분 의미가 없고, 합치면 SR 44/180 vs π0.5 23/180($p = 0.006$)입니다. Score 기준 18과제 중 13과제에서 최고(동률 포함)입니다.
- Wuji 손(Table 8): 합친 ID 27/40 vs 12/40, OOD 40/65 vs 19/65는 유의합니다. 과제별로는 8개 비교 중 2개(Put Away Clothes OOD, Twist off Bottle Cap OOD)만 $p < 0.05$입니다. Collect Shuttlecocks OOD는 점수 분모가 54와 57로 달라 시행 조건이 맞지 않습니다.

추가 검증 (2026-11-11)

추가 검증 1. 학습된 작은 MLP로 Fig. 9 경로 17개 평가 (`extra_learned_denoiser.py`)

- **목적:** §2.2-2.3은 이상적 사후 평균 denoiser로 순서를 봤고, §2.3의 "행동 스트림 최대 step과 성공률의 상관($\rho = -0.82$)"을 "학습된 행동 denoiser가 큰 step에 약하다"로 읽었습니다. 이 해석이 실제로 학습된 모델에서 나타나는지 확인합니다.
- **설정:** `noise_toy.py` 의 세계(봉우리 4개, 영상 8차원, 행동 4차원, base 폭 0.15), 경로 식, Fig. 9 값을 그대로 씁니다. 학습 데이터는 두 스트림의 σ를 독립으로 균등 추출한 결합 flow matching입니다(목표 $\varepsilon - x_0$, 코드 부호). 모델은 2층 tanh MLP(폭 128)이고 입력은 [x_t, σ_v, σ_a]입니다. Adam 1e-3, 배치 512, 15000회, 시드 3개, 손실은 가중치 없는 MSE입니다. 코드의 손실 가중치 w(σ)는 어느 스트림의 σ에 곱하는지 확인하지 못해서 넣지 않았습니다. 평가는 경로마다 10 step Euler, 초기 잡음 2만 개입니다. 지표는 행동이 봉우리 99% 영역 안에 드는 비율입니다.
- **결과 (행동 유효 비율, 시드 3개 평균; sd는 0.002-0.018):**

경로	Fig. 9 (%)	학습 모델	이상적 사후 평균 (§2.2)
동기	93.0	0.770	0.949
행동 선행 α=4 / 8 / 16 / 32	91.0 / 90.7 / 89.7 / 88.5	0.860 / 0.868 / 0.858 / 0.817	0.984 / 0.986 / 0.980 / 0.967
영상 선행 α=4 / 8 / 16 / 32	91.8 / 91.1 / 91.9 / 91.5	0.893 / 0.905 / 0.892 / 0.856	0.992 / 0.994 / 0.993 / 0.985
행동 선행 지연 o=0.2 / 0.4 / 0.6 / 0.8	92.2 / 92.0 / 92.3 / 92.3	0.745 / 0.731 / 0.746 / 0.778	0.927 / 0.897 / 0.861 / 0.832
영상 선행 지연 o=0.2 / 0.4 / 0.6 / 0.8	91.7 / 91.1 / 91.5 / 91.2	0.761 / 0.778 / 0.819 / 0.853	0.942 / 0.930 / 0.926 / 0.917

- 학습 모델의 경로 순서와 Fig. 9의 Spearman은 $\rho = -0.51$($p = 0.037$)입니다. 이상적 사후 평균은 $\rho = -0.45$($p = 0.07$)입니다. 학습 모델이 Fig. 9와 같은 방향의 순서를 낸다는 뜻입니다.
- 학습 모델의 절대 유효 비율은 이상적 모델보다 0.06-0.2 낮습니다(학습 MSE 0.53-0.54). 아직 과소적합이 남아 있을 수 있습니다.
- **예비 실행(학습 6000회, 같은 설정)** 에서는 $\rho = -0.31$($p = 0.23$)이었습니다. 학습량에 따라 순서의 세기가 크게 달라졌으므로, 이 순서는 강하게 확정된 결과가 아닙니다.
- **경로 통계 검사:**
  - 행동 최대 step과 학습 모델의 유효 비율의 Spearman은 $\rho = 0.17$($p = 0.51$)입니다. §2.3이 Fig. 9 성공률에 대해 보인 상관($\rho = -0.82$)은 학습 모델의 유효 비율로 옮겨가지 않습니다. 학습 모델은 "행동 최대 step이 크면 나쁘다"를 보이지 않습니다.
  - 행동 갱신 횟수와 학습 모델의 유효 비율은 $\rho = 0.16$입니다.
  - 행동 선행 α 계열(4개 점) 안에서는 α가 커지면 유효 비율이 떨어지는 경향이 있습니다(α = 8 → 32에서 0.868 → 0.817). 하지만 α=4와 α=8의 차이는 뒤집혀서, 순위 상관은 $\rho = 0.8$입니다(점 4개).
  - 행동이 먼저 가는 선형 지연 계열은 최대 step이 0.1로 모두 같은데도 0.73-0.78로 영상 선행 지연 계열(0.76-0.85, o가 클수록 높음)보다 낮은 쪽입니다. 즉 학습 모델에서는 "큰 step"이 아니라 "행동이 먼저 가는 경로"가 나쁜 쪽에 가깝습니

다.

- **결론:** 학습 모델은 Fig. 9와 같은 방향의 순서를 냅니다. 그러나 그 순서를 만드는 원인은 §2.3이 말한 "행동 최대 step"이 아닙니다. 따라서 §2.3의 해석(학습된 행동 denoiser는 큰 step에 약하다)은 이 toy에서 지지되지 않았습니다. §2.2의 결론(순서는 학습된 네트워크가 평면 영역을 근사하는 방식에서 나온다)은 유지되지만, 그 원인이 스트림 선후 관계인지 step 간격인지는 이 실험으로 가릴 수 없습니다. 경로를 바꿀 때 행동 스트림의 step 간격을 균등하게 두라는 권고는 근거가 약합니다.
- **한계:** 작은 MLP 한 종, 손실 가중치 없음, 시드 3개, 봉우리 4개의 2차 toy. 논문의 실제 모델은 DiT 백본과 많은 데이터로 학습되었습니다. 학습 모델 순서의 세기가 학습량에 따라 바뀌었으므로(위 예비 실행), 실제 모델에서 같은 순서가 나오는지는 검증되지 않았습니다. 성공 지표는 "봉우리 안 행동 비율"이라 실제 RoboTwin 성공률과 다릅니다.

04. 코드 검증: 논문 ↔ 구현 대조, 수치 재계산, 구현 가능성

- **대상:** `OpenWAM-Official/OpenWAM` @ `a00c4ba` (2026-11-02, shallow clone, `third_party/cosmos-predict2.5` 서브모듈은 받지 않음). Hugging Face `OpenWAM/` 모델 46개·데이터셋 9개는 메타데이터, config, safetensors 헤더(가중치 아님)만 받았습니다. `XPolicyLab/XPolicyLab`은 API로 HEAD `6baccd5`만 확인했습니다.
- **범위:** GPU 실행, 가중치·데이터 다운로드, 패키지 설치 없이 코드 읽기, 설정 대조, CPU 재계산만 했습니다.

1. 모듈 매핑

논문	구현 위치	일치 여부
C(E, S, M) 조합, 레지스트리 (§3.1)	`openwam/model/architectures/{single_system,dual_system,tri_system}/`, `configs/model/*.yaml`	✅ 구조 7종이 모두 있습니다
백본 실행 계약 prepare / block step / finalize (§3.1)	`architectures/base.py`의 `BaseWAMArchitecture`	✅
마스크 4종, 영상 first-frame causal (Eq. 1)	`attention_mask_mode: mutual`, `video_attention_mask_mode: first_frame_causal` (`dual_system.yaml`, 사전학습 config)	✅
영상 백본 5종 (§3.1)	`configs/model/video_backbone/` (wan21_vace_1_3b, cosmos_predict25_2b, cosmos3_edge, wan22_ti2v_5b, wan21_i2v_14b_480p)	✅ Cosmos3-Edge는 Fig. 6에 결과가 없지만 HF에 ablation 체크포인트가 있습니다
ActionDiT 1B, 폭 1024 (§5.1.1)	`configs/model/action_backbone/separate_action_dit.yaml` (dim 1024, ffn 4096), 헤더 합계 1.021B	✅
결합 flow matching, 독립 timestep (Eq. 2)	`base.py` 1089행(영상), 1113행(행동)에서 각자 `torch.randint`	✅ 부호 규약은 D4
유효 마스크 m, 조건 프레임 제외	`base.py` 1206-1215행(첫 latent 제외), 1288-1297행(행동 유효 칸 평균)	✅ 배치 평균 방식은 D6
종 모양 가중치, shift 5 (§5.1.2)	`action_backbone/scheduler.py: _set_training_weight`, `shift_video: 5.0`, `shift_action: 5.0`	✅ 식은 D5
동기·비동기 경로 (Eq. 3)	`openwam/deploy/denoise_schedule.py`	✅ 항등식으로 구현(`03` §1.3)
추론 모드 동기·비동기 (§3.3)	`openwam/deploy/executors/`, `inference_mode: sync`	✅
가속 4종 (§3.3)	`configs/deploy.yaml`: prompt 캐시, `decode_video: false`, `compile.enabled: true`, `dit_cache` (cosine 0.99, 최대 3회 연속 skip)	✅ 평가 설정은 D3
WebSocket 서버, 정규화 기록 (§3.2, §3.4)	`openwam/deploy/server.py`, `policy.py`, `model_loader.py`, 체크포인트의 `normalization_stats.npy`	✅
80D 통합 행동 공간 (Eq. 4)	`map_to_unify` (정규화 뒤 scatter), 로더별 `_unify_dst_index`	✅
사전학습 혼합, 소스별 시간 예산 (§5.2.1)	`openwam/dataloader/mixture.py` (`weight_strategy: proportional`, `total_hours`)	⚠️ D1

논문	구현 위치	일치 여부
벤치마크 어댑터 8종 (§3.4)	`benchmarks/{libero,libero-plus,vlabench,robotwin,robodojo,robocasa365,ebench,robocasa_gr1}/`	✅ RoboDojo 평가는 XPolicyLab을 거칩니다

2. 불일치와 논문에 적히지 않은 구현 요소

#	내용	영향
D1	**사전학습 혼합을 공개 코드로 만들 수 없습니다.** HF 사전학습 config는 소스 다섯 개 중 첫째를 `type: worldengine`으로 지정하는데, 데이터셋 레지스트리(`dataloader/registry.py`)에 이 타입이 없습니다. Study(§4.3)에서 쓴 EgoDex 로더도 테스트의 mock에만 있습니다. 공개 `configs/dataloader/mixture.yaml`은 로봇 소스 4개만 있고 `total_hours`가 없어 전체를 비율대로 씁니다. 소스별 `dataset_dir`는 작성자 서버의 절대 경로입니다.	사람 영상 30% 없이는 OpenWAM-α 사전학습도, Fig. 10-11의 ego 변형 학습도 재현할 수 없습니다. 대신 Study 사전학습 변형 5개와 그 SFT 7개의 체크포인트는 공개되어 있어 **평가는 재현할 수 있습니다.**
D2	공개 사전학습 체크포인트는 `checkpoint_step_154000`이고, Table 9는 155,862 iteration(1 epoch)입니다.	98.8% 지점의 체크포인트입니다. SFT 체크포인트 13개의 step은 Table 10과 모두 일치합니다(C4).
D3	**평가 설정이 벤치마크마다 다릅니다.** RoboTwin은 `deploy.yaml` 기본값(velocity 캐시 on, compile on), 과제당 100 episode, seed 0입니다. 공식 README는 "캐시를 끄면 수치가 바뀐다"고 적었습니다. EBench는 캐시 off, compile off로 보고했습니다. LIBERO는 과제당 50회(seed 42), LIBERO-Plus는 과제당 1회, EBench는 26과제 × 20 episode(두 과제는 15), RoboDojo는 3 seed × 50 episode입니다.	논문 §5.1.3은 동기 모드라서 결과가 "모델 자체의 성능을 가장 직접 반영한다"고 했지만, velocity 캐시는 forward를 건너뛰는 근사입니다. 캐시가 RoboTwin 점수에 주는 영향은 공개되지 않았습니다. 재현할 때 README의 벤치마크별 설정을 그대로 써야 합니다.
D4	부호 규약이 논문과 반대입니다. 코드는 σ = 1이 noise이고 목표가 ε − x, 논문은 t = 0이 noise이고 z − ε입니다.	수학적으로 같습니다(`03` §1.1). 코드를 읽을 때만 주의하면 됩니다.
D5	"종 모양 가중치"의 실제 식은 $\exp(-2(\sigma - 0.5)^2)$에서 grid 최솟값을 뺀 것입니다. **순수 noise(σ = 1)에서 가중치가 0**입니다. shift 5 표본추출과 합치면 표본의 36%가 σ ≥ 0.9인데 가중 손실은 10%입니다.	논문에 식이 없습니다. 고noise 쪽에 표본을 몰아 주면서 손실은 깎는 조합이라, 다른 구현으로 옮길 때 그대로 맞춰야 합니다.
D6	**사람 영상 표본도 행동 손실의 배치 평균 분모에 들어갑니다.** 표본별 행동 손실은 유효 칸 평균이고(`clamp(min=1)`), 유효 칸이 없는 사람 영상은 0이 되어 `.mean()`에 그대로 들어갑니다(`base.py` 1296-1297행).	사전학습의 실효 $\lambda_a \approx 0.70$입니다(표본의 30%가 사람 영상). 논문은 $\lambda_a = 1$이라고 적었습니다(Table 9). SFT는 1입니다. 사람 영상 비율을 바꾸면 행동 손실 크기도 함께 바뀝니다.
D7	비동기 경로에서 코드는 α와 o를 **동시에** 줄 수 있습니다(선행에 α, 후행에 o). 선행 곡선은 σ 좌표의 f_{1/α}로 계산합니다.	논문은 두 계열을 따로 실험했습니다. 식은 같습니다(`03` §1.3).
D8	공개 `configs/train.yaml` 기본값은 GPU당 batch 16, 5 epoch, RoboTwin 로더, gradient checkpointing on입니다. 사전학습 config(HF)는 GPU당 24, 1 epoch, gradient checkpointing off입니다.	사전학습 설정은 HF 체크포인트에 딸린 `config.yaml`에서 가져와야 합니다. H200(141 GB)에서 checkpointing을 끈 batch 24는 80 GB GPU에서 그대로 돌지 않을 수 있습니다.
D9	**로더가 소스별 fps를 맞추지 않습니다.** 창은 원 fps 기준 33프레임, 행동은 32 step이고, 영상만 stride 4로 줄입니다(`robotwin.py`, `lerobot_v3_reader.py`).	같은 chunk가 30 fps 소스에서는 1.07초, AgiBot(15 fps) 2.1초, DROID(10 fps) 3.2초입니다. 80D 슬롯 의미는 같아도 시간 축 의미가 소스마다 다릅니다. 논문에는 언급이 없습니다.

#	내용	영향
D10	1 epoch 정의가 맞지 않습니다. 학습 창은 매 프레임에서 시작할 수 있고(최소 창 길이 1-2, 끝은 마지막 프레임 반복) 창 수 ≈ 프레임 수입니다. 그런데 155,862 × 3,072 = 478.8M으로 518.5M 프레임보다 7.7% 적습니다.	원인을 코드에서 찾지 못했습니다. 사람 영상 로더(비공개)의 창 규칙이 다르거나 Table 4 프레임 수 정의가 다를 수 있습니다.
D11	OpenWAM-Alpha 체크포인트 14개(사전학습, SFT)가 모두 고정된 umT5-xxl(5.68B)과 VAE(0.70B)까지 담은 24.81 GB 한 파일입니다(ablation 32개의 크기는 확인하지 않음). 학습되는 부분은 6.02B입니다(C3).	추론 메모리의 절반 가까이가 매 요청마다 쓰지 않는 텍스트 인코더입니다(prompt 캐시가 있으므로). 저메모리 기기에서 가장 먼저 덜어 낼 부분입니다(§4).
D12	실행 환경은 CUDA 중심입니다. README는 PyTorch 2.7.1 + CUDA 12.8, Docker CUDA 이미지를 권하고, 기본 장치는 `cuda:0` 입니다(`--device` 로 바꿀 수 있음). 주의 구현은 flash-attn 3/2, SageAttention, xformers 중 설치된 것을 쓰고, 없으면 torch SDPA로 내려갑니다. Cosmos 백본은 `transformer-engine` 컴파일(nvcc)이 필요합니다. MPS 경로는 코드에 없습니다.	맥에서는 공식 경로로 실행할 수 없습니다. 수정 후 가능성은 §4.2에 있습니다.
D13	RoboDojo 시드별 결과 파일 (`assets/robodojo_verification/`)과 검증 스크립트가 공개되어 있습니다.	이 파일로 Table 17의 OpenWAM-α 행 14칸을 정확히 재현했습니다(C8). 다른 벤치마크에는 episode 단위 결과 파일이 없습니다.
D14	사전학습 config의 소스별 `total_hours` 는 사람 영상과 InternData-A1이 1439.0268로 같고, 합이 6,365.0시간입니다(Table 4는 6,369).	30% 프레임 목표를 시간으로 바꾼 값이라 일치합니다. 표는 실제로 뽑힌 양입니다(`03` §4.1).

3. 수치 재계산 (`analysis/table_check.py` , `analysis/noise_toy.py`)

#	주장 (출처)	재계산	판정
C1	518.5M 프레임, 6,369시간, 30/40/30 (Table 4, §5.2.1)	합계 일치. 사람 30.03%, 실제 로봇 39.98%, 합성 29.99%. 프레임/fps는 RoboCOIN Full(967.6 vs 956시간)만 1.2% 어긋남	✅ (RoboCOIN Full 소폭 불일치)
C2	128 H200 × 7일, 155,862 iteration (Table 9)	478.8M 클립, 21,504 GPU-h, 3.88초/step, DiT 행렬곱 기준 MFU 약 7%(하한)	참고. 1 epoch 정의 불명 (D10)
C3	5B DiT + 1B ActionDiT, umT5·VAE 고정 (§5.1)	safetensors 헤더: DiT 5.000B, Action 1.021B, umT5 5.681B, VAE 0.705B, 합 12.407B(bf16 24.81 GB)	✅
C4	SFT 설정 (Table 10)	HF 체크포인트 파일의 step 13개가 모두 일치	✅
C5	Section 4 표 내부 일관성 (Table 1-2, Fig. 6, 10, 11)	Table 1-2 평균 일치. Fig. 6 5B = Table 2 ASV(92.39). Fig. 11 scratch Mutual = Table 2(92.15). Fig. 11 사전학습 C2R OOD Mutual 26.42 vs Fig. 10 공동학습 26.62	⚠️ 0.20 불일치 1건
C6	Finding 2-3의 비교 (Fig. 9-11)	과제 50 × 100 episode 이항 근사: mutual vs ASV p = 0.29-0.67, 공동학습 vs 2단계 p = 0.89, 동기 vs 최선 비동기 p = 0.06. 큰 효과(구조, 마스크, 사전학습 OOD)는 z > 10	일부만 ✅ (`03` §3)

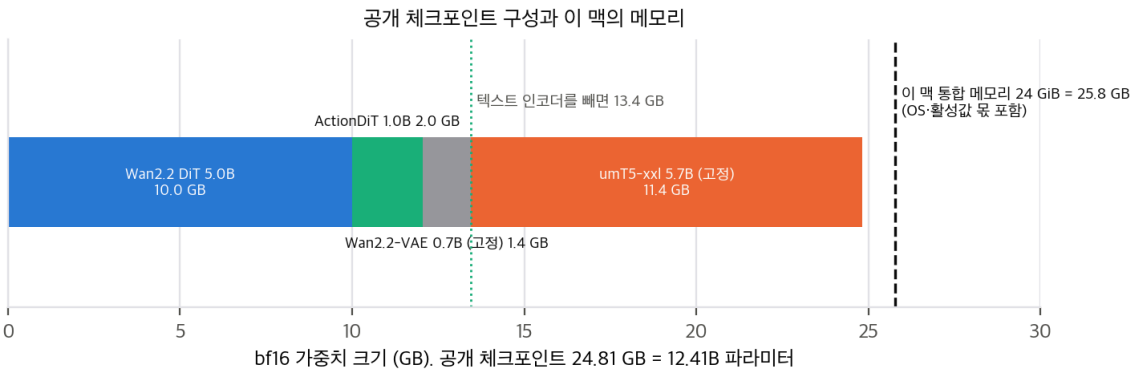
#	주장 (출처)	재계산	판정
C7	부록 D 평균·순위, "최상위권" (§5.3.1)	평균 재현: LIBERO·RoboTwin·VLABench 단순 평균, RoboCasa365 18/16/16, EBench 7/10/9, LIBERO-Plus 가중(RMSE 0.49), RoboDojo 5차원. 1위와 차이: LIBERO −0.1, VLABench −0.2, Full −0.5, GR1 −4.0, RoboCasa365 −19.2	⚠️ RoboCasa365 서술 불일치. VLABench IS 63.42 vs 표 63.5
C8	RoboDojo 11.92 / 17.18 (Table 17)	공개 시드별 파일로 14칸 모두 재현. 시드 SD 0.32, 42과제 중 14과제 성공 0	✅
C9	실물 Table 6-8	합계·평균 일치. Franka vs LingBot-VA p = 0.42. RoboDojo-Real SR 44/180 vs 23/180 p = 0.006. Wuji 합산 p = 0.002 / 0.0004, 과제별 8개 중 2개만 p < 0.05. Shuttlecocks OOD 분모 54 vs 57	✅ 수치, ⚠️ 해석
C10	플랫폼 17개, embodiment 21개, 170 ms (§5.2, §5.1.3)	1 + 15 + 1 = 17, + InternData 4 = 21. Fig. 4 Dual JSA 175 ms와 맞음	✅
V1	"모든 비동기 경로는 학습 분포 안" (§3.2)	index 좌표에서 학습 표본이 균등하므로 표본 밀도는 정확히 맞음	✅ (`03` §2.1)
V2	"동기가 최선" (Finding 2)의 원인	이상적 denoiser에서는 영상 선행이 더 낫고(Spearman −0.45), 실제 순서는 행동의 최대 step과만 상관(−0.82)	경로 성질이 아니라 학습된 모델의 성질 (`03` §2)

4. 구현할 수 있는가

4.1 공식 코드로 논문 결과를 재현하는 경우

단계	필요한 것	판단
공개 SFT 체크포인트 평가 (Table 13-20)	CUDA GPU 1장(체크포인트 24.8 GB를 올리고 활성값 여유. 논문 측정은 RTX 5090 32 GB), 벤치마크별 시뮬레이터 설치, 공식 README의 평가 설정	**가능.** 코드·체크포인트·규약이 모두 있습니다. RoboTwin 전체 평가는 README 표현으로 "모드당 여러 GPU-hour(many GPU-hours)"입니다
Study ablation 재평가 (Table 1-2, Fig. 6-7, 10-11)	위와 같고, 체크포인트 32개 × RoboTwin 조건 2개	**가능.** 비용은 크지만 seed를 늘려 `03` §3의 "잡음 범위" 비교를 다시 가를 수 있는, 이 논문에서 가장 가치 있는 재현입니다
SFT 재현 (Table 10)	사전학습 체크포인트 + 벤치마크 데이터(HF), 80 GB GPU 8장 권장(README)	**가능.** 예: LIBERO 10,690 step × batch 256
Fig. 9 경로 실험	공개 Dual JSA 체크포인트 + `denoise_mode: async` 설정만 바꿔 평가	**가능.** 재학습이 필요 없습니다
사전학습 재현 (Table 4, 9)	사람 영상 1,442시간(비공개) + 로더(비공개), 공개 로봇 데이터 약 9.5 TB, H200 약 21,500 GPU-h	**불가.** 공개 로봇 4개 소스만으로는 다른 혼합이 됩니다 (D1)

4.2 이 맥(Apple M4 Pro, 통합 메모리 24 GiB, MPS, CUDA 없음)



우리 분석: 공개 체크포인트 구성(safetensors 헤더 기준)과 이 맥의 메모리.

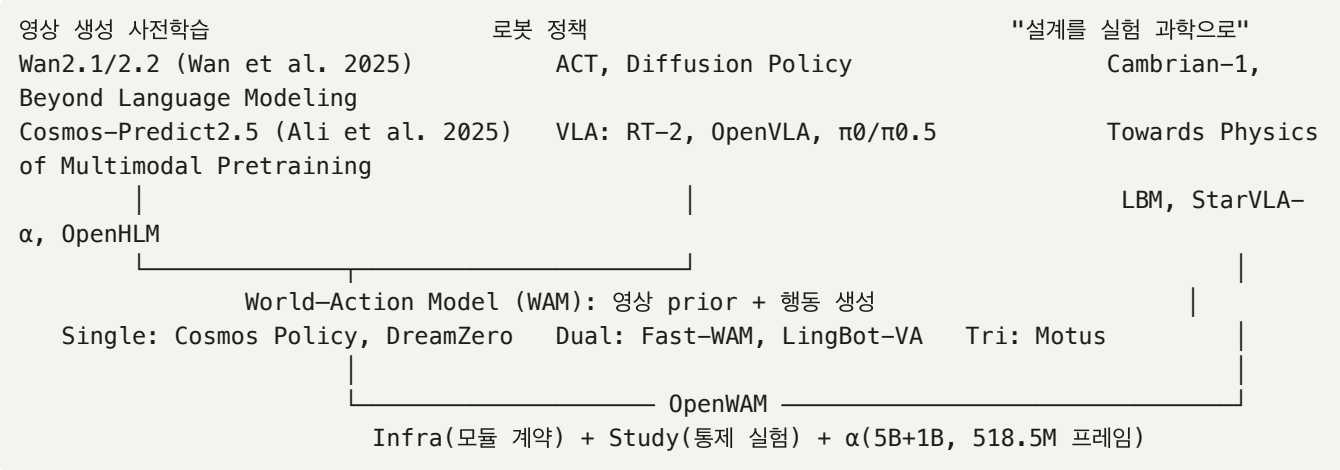
작업	가능 여부	근거
표·그림 재계산, 경로·학습 분포 toy	✅ 완료	`table_check.py` (약 1초), `noise_toy.py` (약 1분), 단일 스레드 numpy
공식 코드 그대로 추론	❌	CUDA 기본 경로, compile + CUDA graph, 체크포인트 24.81 GB가 통합 메모리 24 GiB(25.8 GB)를 OS와 나눠 써야 하므로 들어가지 않습니다
수정한 추론(MPS)	△ 미검증, 조건부	텍스트 인코더 키를 빼고 읽으면 DiT + ActionDiT + VAE가 13.4 GB입니다. prompt 임베딩은 umT5만 따로 한 번 돌려 저장하면 됩니다(서버에 prompt 캐시가 이미 있음). `--device mps`, `--compile-enabled false`, SDPA 주의로 실행 경로는 있어 보입니다. chunk 하나에 5B DiT forward 10회(토큰 약 400개)라 대략 수-수십 초로 추정합니다. LIBERO 몇 episode 확인은 할 만하지만 벤치마크 전체 평가는 비현실적입니다. torch 설치와 체크포인트 25 GB 다운로드 승인이 먼저 필요합니다
학습·SFT	❌	학습되는 파라미터 6.02B에 AdamW 상태만 70 GB 이상입니다
아이디어 축소 재구현	✅ 가능	독립 timestep 결합 flow matching + 마스크 4종 + 경로 17종을 작은 MLP·transformer로 toy 데이터에서 시험하는 것은 numpy·CPU 또는 torch(MPS)로 충분합니다

4.3 판단

구현할 수 있는가: 평가와 SFT는 공식 코드로 CUDA GPU에서 재현할 수 있고, 사전학습은 재현할 수 없습니다. 이 저장소의 WAM 논문 가운데 공개 범위가 가장 넓습니다. 코드, 설정, 사전학습·SFT·ablation 체크포인트 46개, 벤치마크 데이터가 모두 있습니다. 하지만 사전학습 프레임의 30%인 사람 영상과 그 로더가 없고, 계산량이 H200 약 21,500 GPU-h입니다. 이 맥에서는 학습이 불가능합니다. 추론은 공식 경로로는 불가능하고, 텍스트 인코더를 덜어 내고 MPS로 돌리는 수정이 필요합니다(미검증). 이번 작업의 CPU 재계산과 toy는 끝냈습니다.

02. 선행연구와 위치

1. 계보



저자들의 문제의식(§1, §2)은 WAM의 핵심 가설인 "세계 예측과 행동 생성의 시너지"가 단일 시스템 안에서 구조, 학습, 데이터, 샘플링과 얽혀 있어 검증되지 않았다는 것입니다. VLA 쪽에는 OpenVLA, StarVLA, XPolicyLab 같은 열린 생태계가 있는데 WAM 쪽에는 없다는 점도 듭니다.

2. 구성 요소별 출처

구성 요소	출처	OpenWAM에서 바꾼 점
영상 DiT	Wan2.2-TI2V-5B (Wan et al. 2025)	첫 프레임 clean 고정, 토큰별 timestep, 30층 모두 ActionDiT와 공유 주의
시각 인코더	Wan2.2-VAE(48채널, 시간 4배·공간 16배 압축)	고정. Study에서 FLUX.2-VAE, DINOv3, V-JEPA 2.1(+S-VAE)과 교체 비교
S-VAE 어댑터	Zhang et al. 2025a, Jha et al. 2026	표현형 특징을 48차원으로 축소
텍스트 인코더	umT5-xxl (Wan 계열 기본)	고정, 문맥 끝에 고유수용 상태 토큰 1개 추가
ActionDiT, joint self-attention	Fast-WAM, LingBot-VA, Motus 계열의 MoT형 결합	폭 1024, 30층, 공유 공간 24 × 128
결합 flow matching, 독립 timestep	π0 계열 flow matching, Wan의 shift 5 timestep warp	두 스트림 독립 표본, 유효 마스크 m
비동기 denoising 경로	Latent Forcing (Baade et al. 2026, arXiv:2602.11401)의 분산 이동	선형 지연을 추가하고 동기와 같은 코드 경로로 통합
velocity 캐시	DreamZero (Ye et al. 2026c)의 단계 간 재사용	두 스트림 cosine 유사도 기준, 연속 skip 상한
통합 행동 공간	(자체) 80D = 34D 팔 × 2 + 예비 12	고정 슬롯 의미, 정규화 후 scatter
데이터 정제	Qwen-RobotManip (Yuan et al. 2026a) 정제 파이프라인	상태 기반 정지 구간, 시각 교차 확인 규칙 추가
평가 클라이언트	각 벤치마크 공식 코드, RoboDojo는 XPolicyLab	WebSocket 단일 규약
코드 기반	DiffSynth-Studio 계열(주의 구현 환경변수 `DIFFSYNTH_ATTENTION_IMPLEMENTATION`)	백본 계약과 조합 규칙으로 재구성

3. 이 저장소의 다른 논문과의 관계

논문	관계
AW-0 (`agentic_wam/`)	가장 가까운 비교 대상입니다. AW-0도 Cosmos-Predict2.5-2B 영상 전문가(VAE는 Wan2.2로 교체)와 행동 전문가를 MoT로 묶은 Dual-System WAM에 코드 전문가를 더한 Tri형 구조입니다. AW-0 분석에서 "4단계 추론에서 행동이 읽는 미래 영상은 σ ≥ 0.625인 흐린 상태"라고 지적했는데, OpenWAM Fig. 9의 행동 선행 o = 0.8(영상 σ ≥ 0.83에서 행동 완성, 92.3%)이 같은 현상을 통제 실험으로 보여 줍니다. 추론 때 미래 영상의 기여는 작고, 이득은 학습 때 생깁니다(`03` §2). AW-0의 "계획 스트림 + ViT" 효과 분해와 OpenWAM Table 1의 Tri vs Dual(+0.24%p, 유의하지 않음)도 같은 방향입니다. 추가 스트림의 이득은 작거나 통제되지 않았습니다. 두 논문 모두 RoboTwin 2.0으로 평가하지만 AW-0는 리더보드보다 약한 자체 재현 기준선을 썼고, OpenWAM은 공식 리더보드 수치를 그대로 옮겼습니다(Table 15-16).
Hydra-0 (`hydra0/`)	Hydra-0는 행동을 이미지 평면의 점 궤적으로 표현해 영상 모델의 조건으로 씁니다. 정책이 아니라 world model(시뮬레이터) 쪽입니다. OpenWAM의 80D 통합 행동 공간은 반대로 말단 좌표계의 고정 슬롯입니다. 둘 다 "embodiment 공통 행동 표현" 문제를 다르게 풉니다. Hydra-0 분석에서 지적한 "같은 백본 통제 비교 부재"를 OpenWAM은 Infra로 해결하려 했고, Table 1-2처럼 구조만 바꾼 비교를 실제로 냈습니다.
ThinkWorld (`thinkworld/`)	ThinkWorld의 이득 63%가 "1초 뒤 DINOv3 특징 예측" 보조 손실에서 나왔습니다. OpenWAM Fig. 7은 DINOv3 + S-VAE latent를 예측 대상으로 써도 Wan2.2-VAE와 거의 같다(90.18 vs 90.30, 차이 유의하지 않음)는 것을 보입니다. 두 결과를 합치면 "미래 예측 대상이 픽셀일 필요는 없다"는 결론이 강해집니다. LIBERO-Plus Avg가 과제 수 가중 평균이라는 점은 ThinkWorld 분석에서 쓴 가중치로도 재현됩니다(`04` C7).
Chronos Policy (`chronospolicy/`)	OpenWAM의 데이터 로더는 소스별 fps를 맞추지 않습니다. 33프레임 창과 32-step 행동 chunk가 30 fps에서는 1.1초, 15 fps(AgiBot)에서는 2.1초, 10 fps(DROID)에서는 3.2초입니다(`04` D9). Chronos Policy가 다룬 "물리 시각 대신 index로 보는 정책"의 문제가 다중 소스 사전학습에서 그대로 생깁니다. time-aware RoPE나 시각 기반 리샘플링을 넣을 자리가 분명합니다.
PaceVLA (`pacevla/`)	PaceVLA는 행동 chunk의 시간 해상도를 바꿔 실행 속도를 올립니다. OpenWAM의 비동기 추론 모드(지연 d, 실행 n, 버림)와 velocity 캐시는 추론 지연을 숨기는 다른 축입니다. OpenWAM-α 평가는 모두 동기 모드라서, 비동기 모드가 실제 성능에 미치는 영향은 Fig. 9(경로)와 별개로 측정되지 않았습니다.

"체계적 WAM 사전학습"이 위 논문들에 더하는 것

1. **같은 코드에서 바꾼 하나의 변수.** 위 다섯 논문은 각자 구조 하나를 제안하고 기준선과 비교했습니다. OpenWAM은 구조 7종, 마스크 4종, 인코더 6종, 백본 4종, 경로 17종, 사전학습 4종을 같은 trainer·서버·평가 규약에서 돌리고, **ablation 체크포인트 32개를 모두 공개**했습니다. 이 저장소의 다른 논문에서는 "통제 비교 부재"를 한계로 적었는데, 여기서는 GPU만 있으면 그 비교를 다시 평가할 수 있습니다.
2. **효과 크기의 서열.** 재계산하면 서열이 분명합니다. 사전학습의 OOD 이득(+9.3-12.1%p) > 구조 용량(Single → Dual +6.9%p) > 영상→행동 정보 경로(+5.0%p) > 사전학습된 DiT 초기화(+2.1%p) > 백본 크기(5B → 14B +1.4%p) ≫ 마스크 방향, 2단계 vs 공동학습, Tri vs Dual(잡음 범위)입니다(`03` §3). 다른 WAM 논문이 내세운 기여를 이 서열에 놓고 볼 수 있습니다.
3. **"추론 때 미래를 보고 행동한다"는 이야기에 대한 반증 자료.** 경로 실험(Fig. 9)은 WAM 계열 논문(AW-0, DreamZero의 계획-후-IDM 관점)이 암묵적으로 가정한 추론 때 정보 흐름을 직접 시험한 첫 통제 결과입니다.
4. **한계도 같은 곳에 있습니다.** 모든 통제 실험이 RoboTwin 한 벤치마크, seed 1개입니다. 그래서 위 서열의 위쪽 다섯 항목만 믿을 수 있고, 마스크·사전학습 순서에 관한 Finding은 이 설계로는 결론이 나지 않습니다.

4. 공개 저장소와 자산 (2026-11-04 확인)

4.1 GitHub

저장소	역할	라이선스	★	마지막 push	확인한 커밋
OpenWAM-Official/OpenWAM	공식 코드(Infra, 학습, 배포, 벤치마크 어댑터)	Apache-2.0	1,162	2026-11-02	`a00c4ba` (shallow clone, 서브모듈 미초기화)
XPolicyLab/XPolicyLab	정책 평가·배포 생태계, RoboDojo 평가에 사용	Apache-2.0	531	2026-11-03	`6baccd5` (API로 HEAD만 확인, clone 안 함)
Wan-Video/Wan2.2	영상 백본 TI2V-5B, Wan2.2-VAE	Apache-2.0	18,358	2026-09-21	-
Wan-Video/Wan2.1	Study 백본 VACE-1.3B, I2V-14B	Apache-2.0	17,376	2026-03-05	-
nvidia-cosmos/cosmos-predict2.5	Study 백본 Cosmos-Predict2.5-2B, 공식 저장소의 git 서브모듈	Apache-2.0	1,392	2026-06-08	-
modelscope/DiffSynth-Studio	Wan 계열 학습 코드 기반(추정, 환경변수 이름 기준)	Apache-2.0	13,340	2026-11-03	-
starVLA/starVLA	비교 VLA, 모듈형 VLA 플랫폼	NOASSERTION	3,904	2026-10-31	-
yuantianyuan01/FastWAM	비교 WAM(Dual), 표현 학습 가설	NOASSERTION	1,641	2026-08-20	-
Robbyant/lingbot-va	비교 WAM(Dual), 실물 비교 대상	Apache-2.0	1,973	2026-07-09	-
Physical-Intelligence/openpi	π0, π0.5 기준선	Apache-2.0	14,526	2026-08-24	-
RoboTwin-Platform/RoboTwin	Study 평가 벤치마크	MIT	3,038	2026-09-24	-

4.2 Hugging Face 모델 (`OpenWAM/`, 46개, 모두 Apache-2.0 태그, 메타데이터만 확인)

묶음	개수	내용	확인한 값
OpenWAM-Alpha-Pretrain-Foundation-Model	1	사전학습 체크포인트	`checkpoint_step_154000.safetensors` 24.81 GB, sha `52df4e6`, 2026-09-09
OpenWAM-Alpha-Sim-*	8	LIBERO, VLABench, RoboTwin-Full, RoboTwin-Clean2Random, RoboDojo, RoboCasa365, EBench, RoboCasa-GR1	파일 이름의 step이 Table 10과 모두 일치 (10690, 6000, 118655, 10740, 60000, 60000, 100000, 100000)
OpenWAM-Alpha-Real-*	5	Franka, RoboDojo ARX-X5·Piper·Piper-X, Wuji 손	step 9860, 30000 × 3, 10925 (Table 10과 일치)
Study ablation (`robotwin_*`, `pretrain_*`, `sft_*`)	32	구조 7, 마스크 3(+기본), 백본 4, 인코더 6, 사전학습 5, 사전학습 후 SFT 7	파일 크기는 확인하지 않음(백본·인코더마다 다름). OpenWAM-Alpha 14개는 모두 24.81 GB(고정 umT5 포함)

4.3 Hugging Face 데이터셋 (`OpenWAM/`, 9개, 받지 않고 크기만 확인)

데이터셋	크기	HF 라이선스 태그	비고
assets	0.18 GB	MIT	데모 영상 등
LIBERO	1.94 GB	Apache-2.0	SFT용 변환본
VLABench	13.16 GB	MIT	
RoboCasa365	83.45 GB	없음	
RoboCasa_GR1	43.93 GB	없음	
wuji_teleop_data_subset	1.86 GB	없음	실물 손 데이터 일부
AgiBotWorld-Beta	4,004.96 GB	없음	사전학습 소스(정제본). 원본 HF 데이터셋은 gated
InternData-A1	1,389.75 GB	없음	사전학습 소스(정제본). 원본은 gated
Droid	148.12 GB	없음	사전학습 소스(정제본)

- RoboCOIN은 `OpenWAM/` 아래에 없고 원 저장소(`wayrise/RoboCOIN`, 약 3,929 GB)를 씁니다.
- **1인칭 사람 영상(WorldEngine, 사전학습 프레임의 30%)은 공개되지 않았습니다.**
- 공개된 로봇 사전학습 데이터만 받아도 약 9.5 TB입니다. 라이선스 태그가 없는 정제본은 원 데이터셋 약관을 따로 확인해야 합니다.

5. 가까운 대안

- **Fast-WAM (Yuan et al. 2026b):** Dual-System WAM이고, "WAM의 이득은 추론 때 계획이 아니라 학습 때 표현에서 나온다"는 표현 학습 가설을 냈습니다. OpenWAM Fig. 9는 이 가설을 지지하는 자료로 인용됩니다. 사전학습이 없어 LIBERO-Plus(51.5)와 RoboTwin C2R randomized(1.9)에서 크게 무너집니다.
- **ABot-M0.5 (Chen et al. 2026a):** 같은 Wan2.2-VAE 픽셀 latent WAM이면서 LIBERO, RoboTwin-Full, RoboCasa365에서 OpenWAM-α보다 높습니다. 저자들은 단일 팔 데이터 규모 차이로 설명합니다(Fig. 15).
- **LingBot-VA / 2.0 (Li et al. 2026b, Zhang et al. 2026b):** Dual-System. RoboTwin-Full에서 2.0이 OpenWAM-α와 동률(93.60)이고, Franka 실물에서 OpenWAM-α와 차이가 유의하지 않습니다(99/120 vs 93/120).
- **Qwen-RobotManip (Yuan et al. 2026a):** VLA. C2R, LIBERO-Plus, RoboTwin-Full에서 OpenWAM-α보다 높고, 데이터 정제 파이프라인의 출처이기도 합니다.
- **DreamZero (Ye et al. 2026c), Cosmos Policy (Kim et al. 2026b):** Single-System WAM. Table 1의 Single 계열 약세(85.5, 84.6)는 이 계열의 구조적 한계를 시사하지만, 원 논문 구현과 같은 조건은 아닙니다.